

RiskRadar — ML / Data Engineering Progress Update

#5

Author: Ire (ML / Data Engineer) **Covers:** A correction to updates #1–#4, found via independent cross-checking with Chidera **Status:** Fifth addition — supersedes the detection-capability claims in earlier updates

1. What this update is

This is a correction, not a new feature. Chidera independently ran his own evaluation of the main dataset (`nigerian_transactions_clean.csv`) and reported that fraud labels rank no better than random across every method he tried (PR-AUC 1.00–1.01x random, precision 2.9–4.4% against a 3.6% base rate). His report explicitly recommended that **no detection numbers be quoted from this dataset**.

Rather than accept or dismiss that on its own, it was checked against our own results directly. **His finding is confirmed, independently reproduced, and the mechanism behind it is now fully understood**. This update corrects the record.

2. What was reported before, and why it was misleading

Updates #1–#4 repeatedly cited two numbers as evidence of a strong fraud signal:

- `is_ato_risk` (`new_device_transaction`) feature importance: **36.5%**, the dominant feature in the model
- Fraud rate split: **0.000% when `is_ato_risk` = 0** , **4.386% when `is_ato_risk` = 1** , with **100% recall** achievable from this feature alone

These numbers are real outputs of real code — they were not fabricated — but they were the wrong metric to lead with, and reporting them as "evidence of a strong signal" was a mistake.

3. The correction, with the exact mechanism

Checked directly: `is_ato_risk` = 1 for **82.07%** of the entire dataset (4,174,079 of 5,086,000 rows). Only 17.93% have `is_ato_risk` = 0 .

Feature importance and recall look impressive because *all* fraud sits inside that 82% majority group — but so does the vast majority of *non-fraud* too. A feature can only rank fraud above non-fraud using information that actually distinguishes the two, and when 82% of everyone (fraudulent and legitimate alike) shares the same value, there is very little left to distinguish with.

Measured directly, worked by hand and confirmed against code:

```
ROC-AUC = P(non-fraud lands in the 0-group) + 0.5 x P(non-fraud lands in the 1-group)
         = 0.186 + 0.5 x 0.814
         = 0.593
```

This is an exact match to what the code computes independently (0.5931), and an exact match to Chidera's reported range. **ROC-AUC of 0.59–0.60 means the model performs only marginally better than random guessing at the actual task of ranking fraud above non-fraud** - even though the same data, viewed through recall or feature-importance, looked strong.

4. Two independent evaluators, two split methods, one answer

To rule out the possibility that this was a quirk of one evaluation approach, the check was run both ways on our own pipeline:

Split method	Full model ROC-AUC	<code>is_ato_risk</code> alone ROC-AUC
Temporal (train on older 70%, test on newer 30%)	0.5959	0.5931
Random stratified (our original approach)	0.5957	0.5930

The two split methods agree with each other to the third decimal place, and both match Chidera's independently-run pipeline. **This rules out split methodology as an explanation and confirms the finding is a real property of the dataset**, found the same way twice, by two different people, using two different codebases.

5. What is retracted, and what is not

Retracted: any claim that the ML model or the `is_ato_risk` rule demonstrates real fraud-detection capability on this dataset. The recall and precision numbers in updates #1, #2, #3, and #4 are real outputs of real, correctly-run code, but should not be quoted as evidence the system catches fraud - they describe what a threshold-based classifier does when most of the population shares one feature value, not genuine detection skill.

Not retracted, and still true:

- The leakage removal (four pre-computed risk/score columns, documented in update #3) was correct and necessary regardless of this finding.
- The shared feature-calculation package (`m1/features.py`), its test suite, and the model contract (update #4) are sound engineering, independent of what this specific dataset's labels turn out to support.
- The rule-by-rule validation discipline (updates #1-#2: testing each rule against real labels, dropping what didn't hold up, disclosing sample-size limits) is exactly the process that made this correction possible in the first place - the same standard applied to every earlier finding is now being applied to itself.

6. What this changes about the project's headline claim

Before: "We built a model that detects fraud with strong recall."

Now: "We built and validated a disciplined fraud-detection pipeline - leakage-free features, a tested training/live-parity contract, honestly validated rules - and, through cross-checking with a teammate's independent evaluation, discovered that our primary dataset's labels do not carry enough real signal to support a genuine detection claim. This was caught methodically, not accidentally, and is disclosed rather than hidden."

This is a defensible, and arguably stronger, story for a capstone defence: it demonstrates the actual ML engineering skill of catching a false positive in your own results, rather than presenting an inflated one.

7. What's next

- **IEEE-CIS validation** (in progress, blocked on Kaggle download): using IEEE-CIS's real chargeback-based labels to test whether the same pipeline achieves a meaningfully higher ROC-AUC when real signal is present. This will be reported as a separate, clearly-labeled validation exercise - never merged with or presented as equivalent to the main dataset's numbers.
- **Lead with ROC-AUC/PR-AUC first** in all future evaluations, before recall/precision, per the lesson of this update.

- Existing progress updates #1-#4 will carry a note at the top pointing to this correction, rather than being silently edited - the paper trail stays intact.